



Loyola Marymount
University

CMSI 5350/EECE 5150 Machine Learning

Professor Lanyu Shang

Announcements

- Homework 1 due this **Friday**
 - Code -> PostCommit (GitHub)
 - Written part -> Gradescope
 - 5-point bonus for early submission by 5pm!
 - PostCommit Quiz next Tuesday

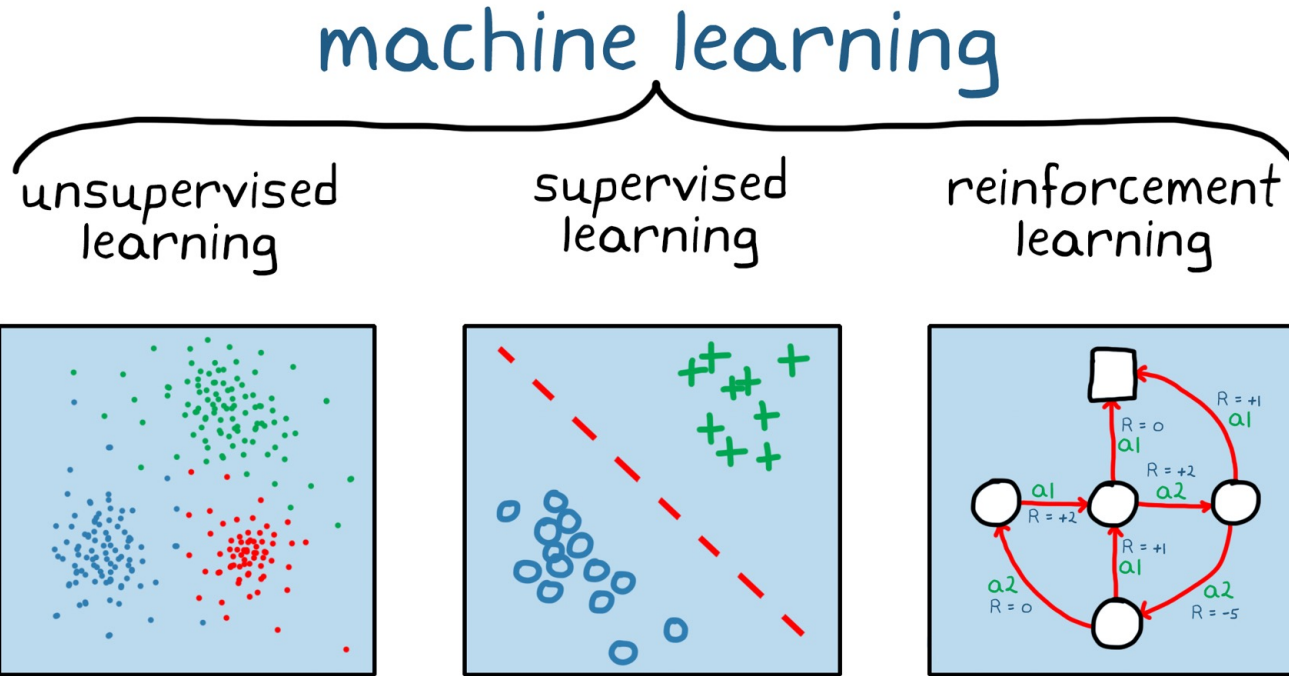
This lecture

- Math formalization of ML
- Decision tree

Next

MATH FORMALIZATION

Types of learning



Learning from data – supervised

- Supervised learning
 - **Input**/Features: X
 - **Output**/Target/Label: Y
- What to “learn” from data?
 - **Goal**: functional mapping from input X to output Y :

$$f: X \rightarrow Y$$

The diagram shows a data table with five columns: Size, Beds, Baths, Zip, and Price. The first four columns are grouped under the label 'Features', and the last column is labeled 'Price'. A bracket on the left side of the table is labeled 'Rows', and a bracket at the bottom is labeled 'Column'.

	Features				Labels
	Size	Beds	Baths	Zip	Price
	1100	1	1	64576	1.29
	1900	3	1.5	78321	2.14
	2800	3	3	98712	3.10
	3400	4	3.5	25721	3.75

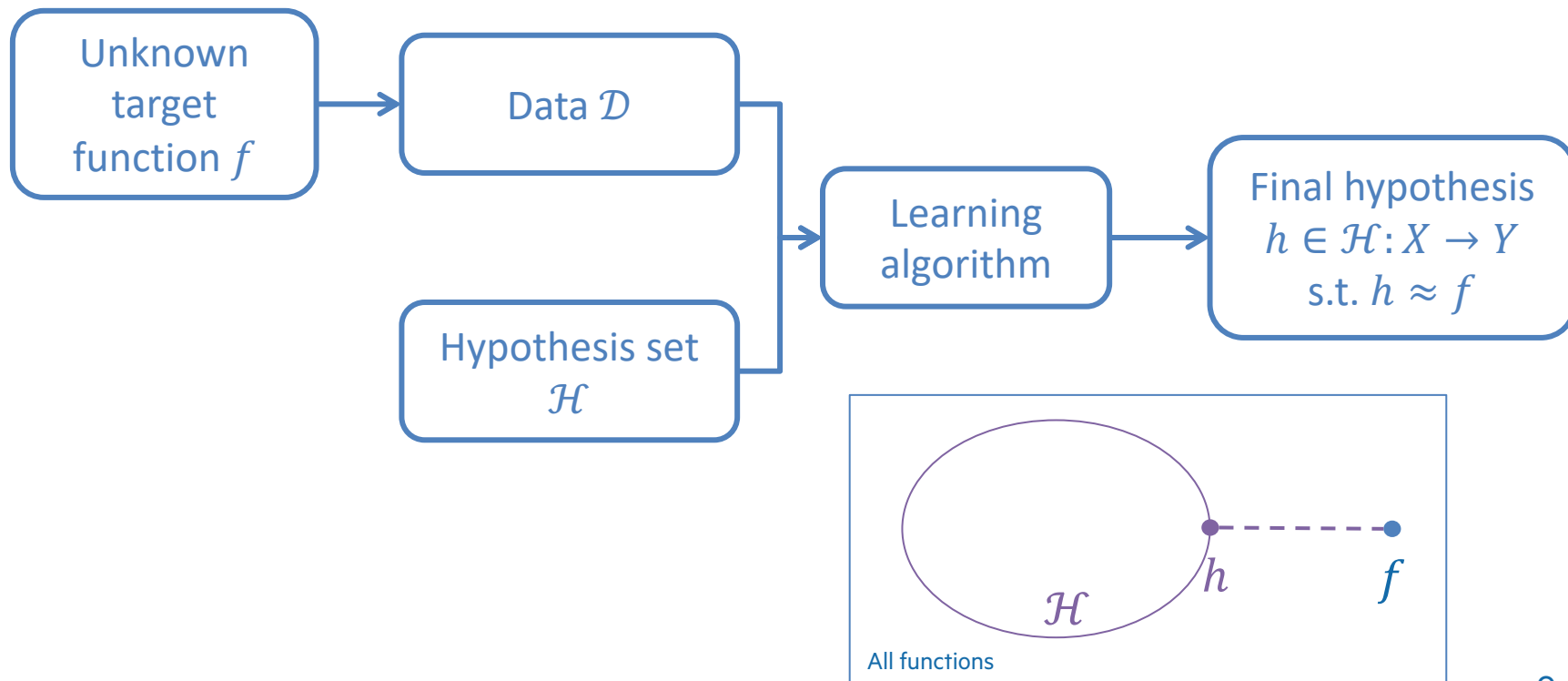
Learning from data – unsupervised

- Unsupervised learning
 - **Input**/Features: X
 - **No explicit targets**
- What to “learn” from data?
 - **Goal**: find structure, representations, or groupings in data

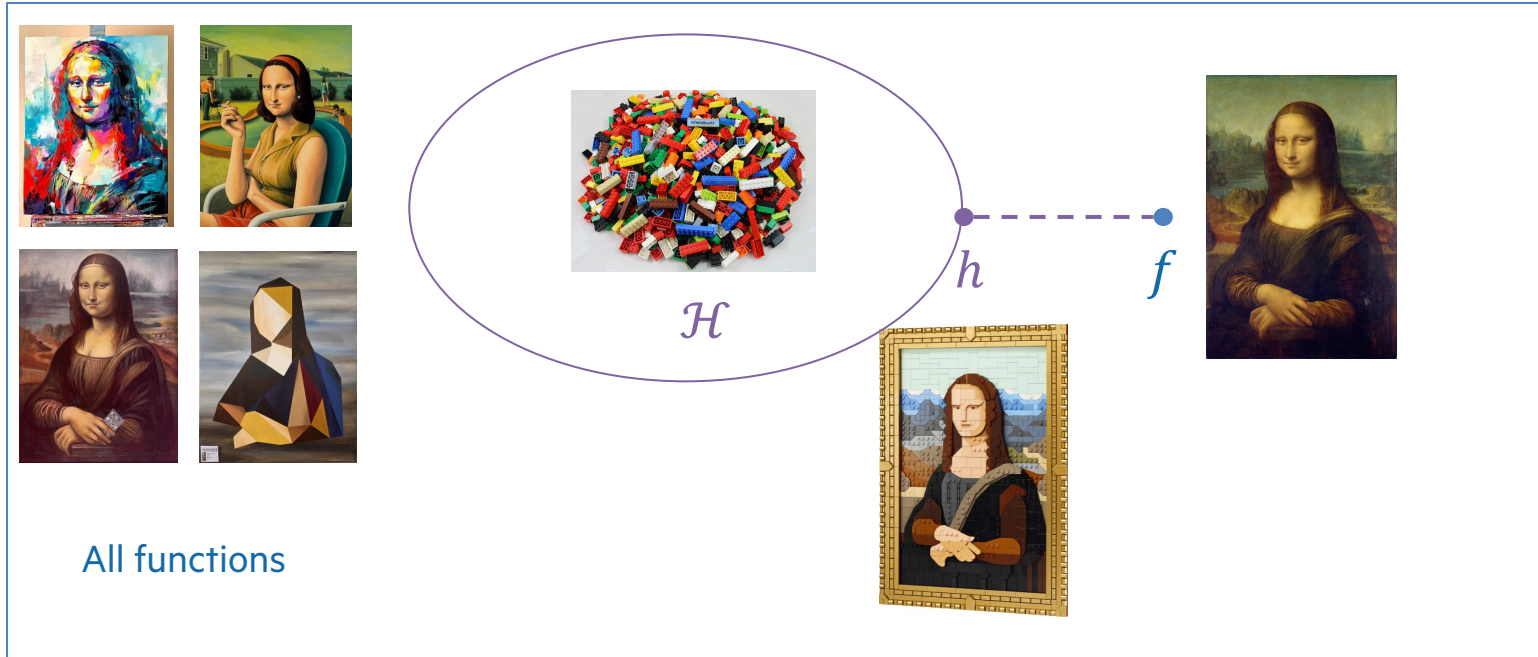
ML as function approximation

- Problem setting
 - Set of possible **instances/samples** X
 - Set of possible **labels** Y
 - Unknown **target/ideal function** $f: X \rightarrow Y$
 - Set of **function hypotheses** $\mathcal{H} = \{h \mid h: X \rightarrow Y\}$
- Input: training data drawn i.i.d. from f
$$\mathcal{D}_n = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\}$$
- Output: hypothesis $h \in \mathcal{H}$ that best approximates f

The learning problem

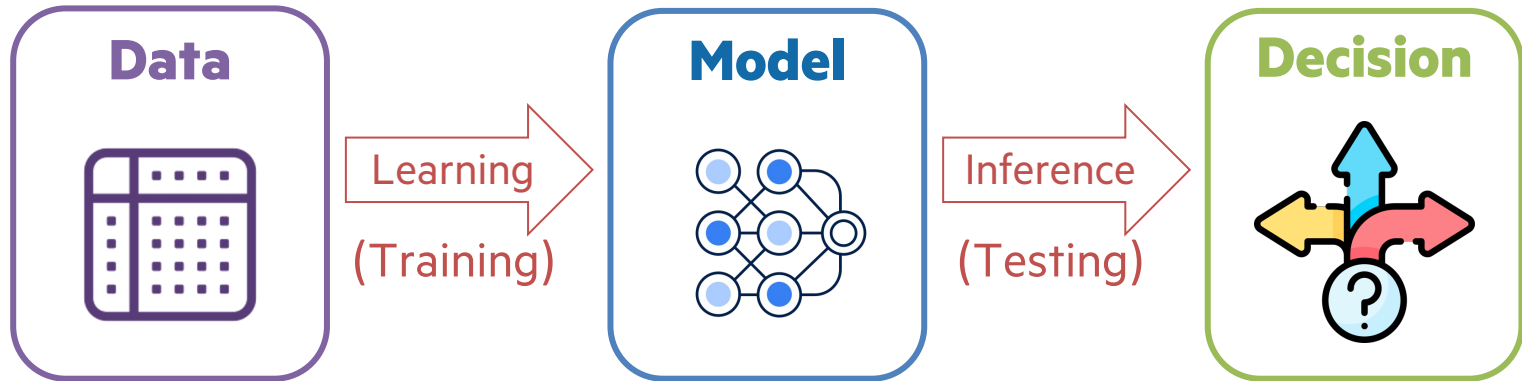


The learning problem



Learning and inference

- Learning: “develop” a model (learn a predictive function)
- Inference: “apply” a model (predict on a new input)

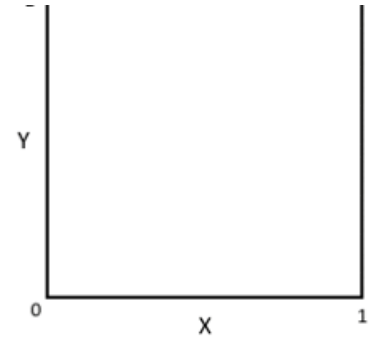


Next

DECISION TREE

What is a decision tree?

- Decision tree is a tree-like model that makes decisions by **splitting data** based on feature values at each internal **node**.



Example - regression

Predictors				Target
Outlook	Temp.	Humidity	Windy	Hours Played
Rainy	Hot	High	False	26
Rainy	Hot	High	True	30
Overcast	Hot	High	False	48
Sunny	Mild	High	False	46
Sunny	Cool	Normal	False	62
Sunny	Cool	Normal	True	23
Overcast	Cool	Normal	True	43
Rainy	Mild	High	False	36
Rainy	Cool	Normal	False	38
Sunny	Mild	Normal	False	48
Rainy	Mild	Normal	True	48
Overcast	Mild	High	True	62
Overcast	Hot	Normal	False	44
Sunny	Mild	High	True	30

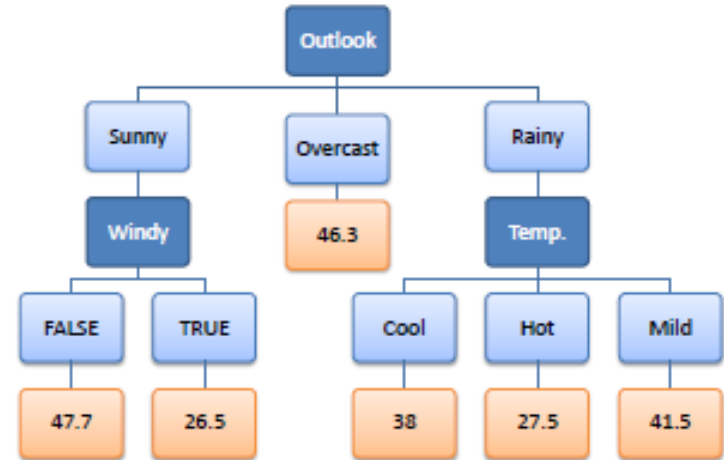
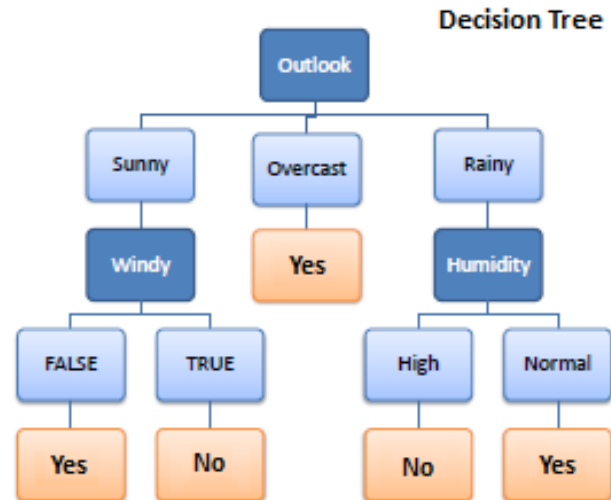


Image source: http://www.saedsayad.com/decision_tree_reg.htm

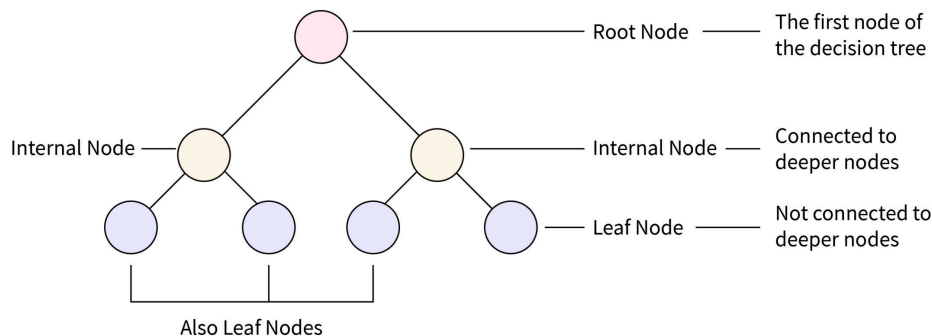
Examples - classification

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



Decision tree – setup

- Structure
 - Root node
 - Internal (split) node
 - Test one feature X_i
 - Leaf (terminal) node
 - Predict y
- We focus on binary trees.



Decision tree – problem setting

- Set of possible instances $X = \{x^{(1)}, x^{(2)}, \dots, x^{(n)}\}$
 - Each instance $x^{(i)} \in X$ is a feature vector
 - E.g. $x^{(i)} = \langle \text{humidity}=\text{low}, \text{wind}=\text{weak}, \text{outlook}=\text{rain}, \text{temp}=\text{hot} \rangle$
- Set of possible labels $Y = \{y^{(1)}, y^{(2)}, \dots, y^{(n)}\}$
 - Y can be discrete, e.g., $y^{(i)} = \{\text{'yes'}, \text{'no'}\}$
 - Y can be continuous, e.g., $y^{(i)} \in \mathbb{R}$
- Unknown target function $f: X \rightarrow Y$
- Set of function hypotheses $\mathcal{H} = \{h \mid h: X \rightarrow Y\}$
 - Each hypothesis h is a decision tree
 - Tree sort $x^{(i)}$ to leaf, which assigns $y^{(i)} \in Y$

Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No

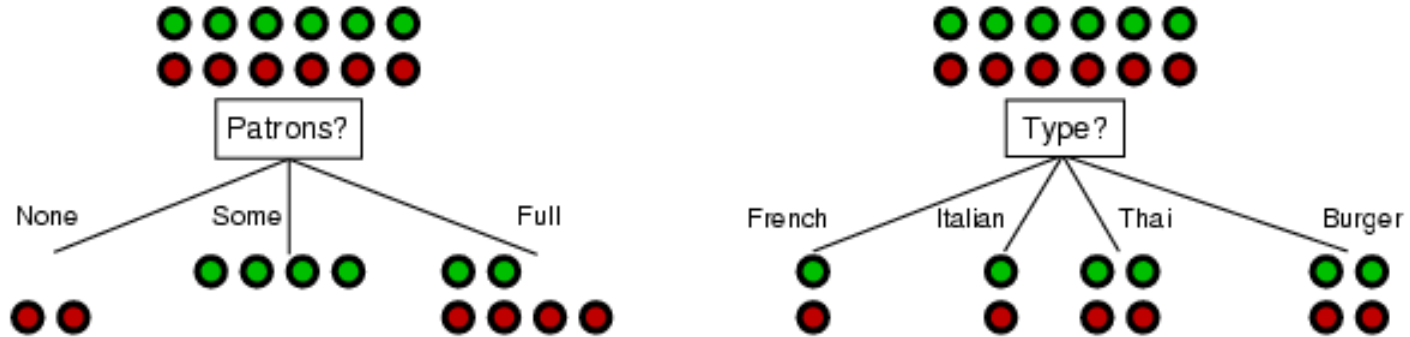
Learning a decision tree

- Finding the optimal tree is computationally intractable.
 - Search space is exponential: 2^m nodes for m binary features
- ML: heuristic search approach (ID3 algorithm)
 1. Start from the **root**
 2. Find the non-terminal node that results in the “**best**” **split**
 3. Create new **descendant of node**, sort/split the data
 4. **Repeat** 2-3 until a stopping criterion is reached

How to find the
“best” split?

Learning a decision tree

- Which split is more **informative**: Patrons? or Type?



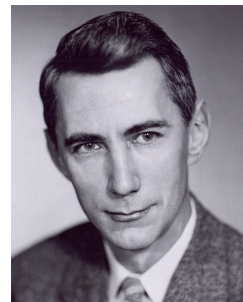
Example: whether to wait for a table a restaurant. Green: yes, red: no

Entropy

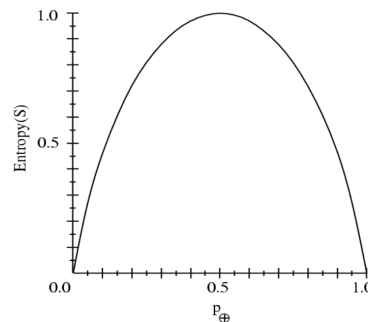
- It measures **uncertainty** of random variable
- Formally, entropy $H(X)$ of a r.v. X is:

$$H(X) = - \sum_{i=1}^n P(X = i) \log_2 P(X = i)$$

- Entropy = 0
 - Perfect purity (all examples belong to one class)
- Entropy = 1
 - Maximum uncertainty for binary classification
- Higher entropy: more mixed/uncertain the dataset



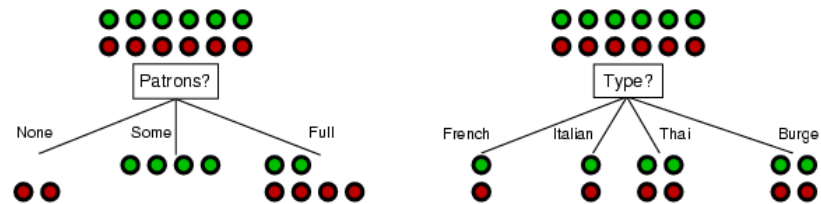
Claude Shannon
(1916-2001)



Binary Classification

Information gain

- Decision tree
 - Goal: reduce entropy on target Y
 - How much do we **gain** from knowing one of the attributes (i.e., what is the entropy reduction)?



- Information gain (a.k.a. mutual information)

$$I(Y, A) = H(Y) - H(Y|A)$$

$$H(Y|A) = \sum_{\alpha \in A} P(A = \alpha) H(Y|A = \alpha)$$

$$H(Y|A = \alpha) = - \sum_{y \in Y} P(Y = y|A = \alpha) \cdot \log_2 P(Y = y|A = \alpha)$$

Example

- Step 1: $H(Y)$

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

$$H(X) = - \sum_{i=1}^n P(X = i) \log_2 P(X = i)$$

Example

- Step 2: info gain between Y and each feature.
 - 2.1 Windy

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

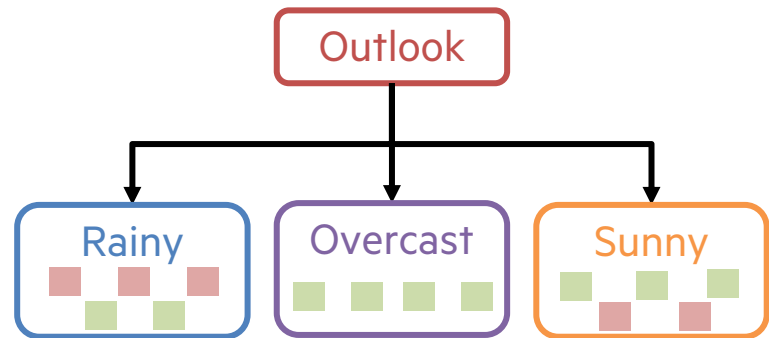
$$I(Y, A) = H(Y) - H(Y|A)$$

$$\begin{aligned} H(Y|A) &= \sum_{\alpha \in A} P(A = \alpha) H(Y|A = \alpha) \\ H(Y|A = \alpha) &= \\ &= - \sum_{y \in Y} P(Y = y|A = \alpha) \cdot \log_2 P(Y = y|A = \alpha) \end{aligned}$$

Example

- Step 2: info gain between Y and each feature.
 - $I(Y, \text{Outlook}) = \mathbf{0.246}$ ✓
 - $I(Y, \text{Temp}) = 0.029$
 - $I(Y, \text{Humidity}) = 0.151$
 - $I(Y, \text{Windy}) = 0.048$
- Outlook** as the root node
- Next, determine the next split for each node in ➡

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



Gini index

- Another approach to measure information gain.

$$1 - \sum_{i=1}^n P(X = i)^2$$

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini',
splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features=None, random_state=None,
max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None,
ccp_alpha=0.0, monotonic_cst=None)
```

[\[source\]](#)

Parameters:

criterion : {"gini", "entropy", "log_loss", default="gini"

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see [Mathematical formulation](#).

BFS or DFS?

- Training vs inference
- DFS, why?
 - Memory efficiency
 - Easy to implement the stopping criteria
 - Intuitive

